

# Dove's Network Acey Dufresne

**Abstract**—The goal of this project is to create a pipeline, for vulnerable people to re-enter the workforce, using several natural language processing methods to make a final prediction of careers, based on the individual users. This proposal will go in depth into the technical requirements and options to implementing the proposed architecture.

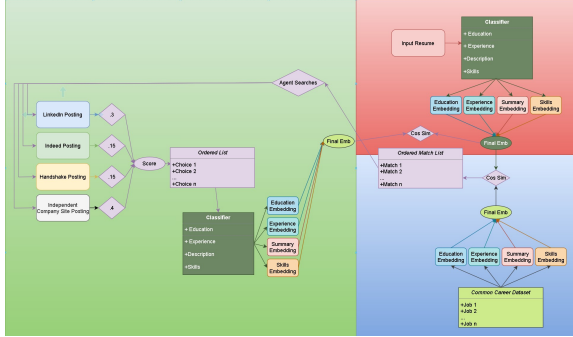


Figure 1. Overview Architecture

## I. OVERVIEW:

\*Flow of System: The architecture can be split into three primary components: The input section, the web crawler/dynamic dataset, and the static dataset. Input Section: In this section the algorithm is being passed an input resume. The input component is acting as a large classifier, breaking down the resume into separate segmentations and assigning a class label to each segment. This is done in two passes:

### A. heuristic approach:

a.) the heuristic pass: Several pre-set datasets: experience, education, professional summary, and location. Each dataset has a corresponding hash set, which contains embedded vectors values representing keywords that are associated with these categories. Each key in the map contains a weight value, which will be applied to the text input. The algorithm converts the resume from a pdf form into text input and then uses a window to 'look' at each line in the input one at a time. Within each line, the algorithm embed each token, or term, and checks if that term is contained in any of the class datasets, if a term matches a class set, then the hash value weight is applied to that window. As can be seen in the 'Term Counting' example, each window is weighed and then categorized.

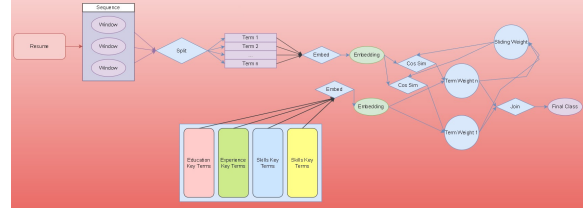


Figure 2. Overview of heuristic section

### B. deep learning pass:

b.) The deep learning pass: After the heuristic approach, each class category that has been sorted from the input resume is compared to the corresponding job listing section using a deep learning model, to find buried patterns between specific sections. This is done because a lot of contexts is lost in the initial heuristic approach. For example, a recently graduated university student, on average, will not have much industry experience, therefore their education acts as experience, using quantitative guide lines, the algorithm will struggle to differentiate between subtle patterns like this. With the deep learning model, the algorithm can learn to identify subtle patterns with semantics, rather than being limited to syntax. This pass is used as an additional weight for after the heuristic approach has already classified and sorted each segment.

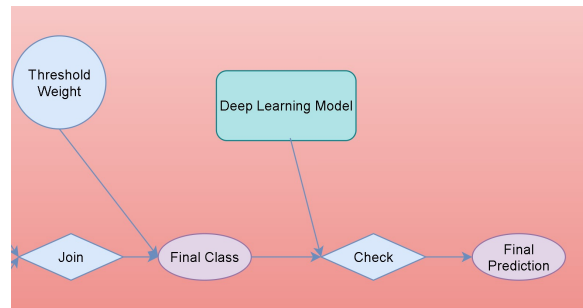


Figure 3. Final threshold weights with Deep Learning network

## II. PRE-PROCESSING:

### A. Input Resumes:

Input Resumes: Tokenized, then converted to lowercase. The TF-IDF scores for each term is found:  $\log(N/1+df) * (N/t)$  Then the input is cleansed of stopwords: " 'a,' 'an,' 'the,' 'and' ." The \*Back-end: heuristic approach can be seen as a form of preprocessing, as each term is sorted into different clusters. Furthermore, the same operation is done on the dynamic web-crawler dataset, consisting of job listings.

### B. Static Dataset:

b.) Static Dataset: <https://www.kaggle.com/datasets/snehaanbhawal/resume-dataset/data>: The Kaggle dataset is a well-documented set that includes over one-thousand example resumes. The TF-IDF scores are calculated for each term, and will then be used for the last layers of the pre-trained deep learning algorithm to train. Five Category Maps: Each term in these sets is embedded before the algorithm begins running, this saves resources when a user is attempting to run the program. Each category is set around one theme: “ ‘education,’ ‘experience,’ ‘technical skills,’ ‘professional summary,’ as well as an additional map for any extra terms.

## III. FRONTEND DESIGN:

### A. Security:

-Security: Since the algorithm is dealing with a large quantity of PII, personally identifiable information, exhaustive security measures will be required. For example, data collected will include: locations of employments, names of Universities attended, personal names of users, writing style and format of user's resume. Additionally, for the users of job boards, “ ‘LinkedIn,’ ‘Handshake,’ ‘Indeed,’ and others: locations of company property, employment compensation, job listing details and requirements. \*Argon2 Passwords: For frontend passkey security, this includes user login information, admin control, and passcodes to access the secure database. \* AES-256 encryption: For any data that must be stored for future AES-256 should be used. This will be data that can be used to improve the algorithm, for example, after a user has entered their resume and the classifier has passed the parsed information to the web-crawler, and the results exceed a pre-determined threshold: then those results should be stored in a set, which can then be added to the pre-existing static dataset, to use to both help train the deep learning algorithm, and for TF-IDF scores for the heuristic approach. \*PostgreSQL storage: After the data has been retrieved and encrypted it is stored in a PostgreSQL database, which is a free software. This creates layers of security, which means breaking one layer does not reveal critical/sensitive information. This will be covered in more depth in \*'System Integration.'

### B. Services:

-Services: This section is not strictly in the scope of the Software Engineering/ML engineering project, but this algorithm is extremely versatile. Because this project will be done primarily in Python, see section \*'Backend Design.Component One.a, due to the necessity of specific Python modules such as the

Playwright API, the Python script(s) will be separate from any service. For example, if the algorithm is integrated into a live site, then the site would handle the information as previously discussed in \*'Security,' and would only pass an environmental variable to a server that hosts the algorithm. In return, the algorithm returns to a global environment variable back to the site, or mobile application. a.) Website: - From scratch: Using Visual Studio Code, the backend can be built. The benefit in this approach is that I have experience in doing this method. The frontend would be built in React, or in VSC using raw HTML, CSS, and JS filing system. Hosting would need to be handled on a different platform, which will be covered in more detail in System Integration. -From WordPress: Using a template based approach, which can be quite cost effective, and reduces the intricacy in building a site. I don't have much experience with this, I have only built one site using WordPress, it has ease of use, but is limited in how a site can be built, in terms of creative. -From Wix: Not a strong suiter, more expensive than WordPress, in my experience. It uses a template based system, but many of the elements needed for a site require a subscription. This leads to a lot of embeddings within the frontend, which slows down performance. As efficiency in the algorithm is already an issue, this would not be a good pairing. b.) Mobile Application: -From Kotlin/KMS: Using Kotlin for Android mobile applications, Kotlin is useful as I have experience with it, and is intuitive to use. IntelliJ has a strong IDE with useful mobile simulations for developing both the frontend and backend operations. The main drawback is that Apple IOS development is much more difficult and requires a lot of conversions to work on Apple services. \*Input: The algorithm requires a PDF resume document, all operations for parsing and preprocessing should occur in the backend, as discussed in the \*'Security,' section. Whatever services is used to host the algorithm only needs to pass the PDF, and two additional variables. These should be optional to the user, for desired location and name. As discussed in the \*'Backend Design,' the algorithm can make a prediction on these variables, but if a user wants to enter them, that makes the task much simpler. Other inputs, that would be useful but not required, are financial goals, preferred companies, and preferred positions. The algorithm will compare what experience users previously have had, but if a user wants to go into a new field or learn new skills, then a weight can be added to skew results for specific careers. \*Output: The algorithm will output around ten-to-twenty by four options. The top twenty listings from LinkedIn, which will then be checked if those listings exist on other platforms, such as Indeed, or Handshake. The backend will then pass back the



has found the job listings, it uses the Beautiful Soup Module to gather the listing details, this includes experience/education/skills requirements as well as location, potential salary/income. The agent uses the same methodology as before and uses a window to classify the windows into their respective categories.

b.) The agent repeats these steps for several other job boards, checking the reputability of the company who owns the posting. The agent first does a search of the company, checking the top five results to see if a site that corresponds to the listed companies name appears, then on the site checking for phone numbers in correlation to a set of terms, for example: “Home Office,” as well as store listings, company reviews, and similar subjects. Each of these subjects will be a separate weight, which will give a combined scoring for each company. Furthermore, the agent will repeat the steps of part a on other job boards, such as: “Indeed,” and “Handshake,” to check if a similar listing is available, which will factor in to the final reliability score. These sets of operations would suit a deep learning algorithm the best, as they will require deep semantic understanding and parsing of text, which would be difficult to accomplish with simple heuristic guides.

c.) Comparison: The agent uses the returned list of categorized cluster from the job listing to compare against the pre-calculated groups of data from the input resume. Then using either cosine similarity or a deep learning model, the algorithm can find the difference between the two sets of data.

#### A. Vector Embeddings (Cosine Similarity):

In this approach, the agent would simply consider and compare the vector embeddings of each category, in their entirety. As can be seen below, from the proof of concept: The results can vary, but obtaining a high similarity score can prove difficult, as the algorithm is comparing term for term. This means that using different synonyms and different orderings of terms can greatly impact precision.

#### B. Beam Search(Tree structure):

In this approach, the agent would find sequences, or sentences in raw form, that began with similar or exact terms, then find the top most probable branches and compare those. As demonstrated below, the idea is to use the initial Kaggle dataset, which contains example resumes, as a reference to compare against.

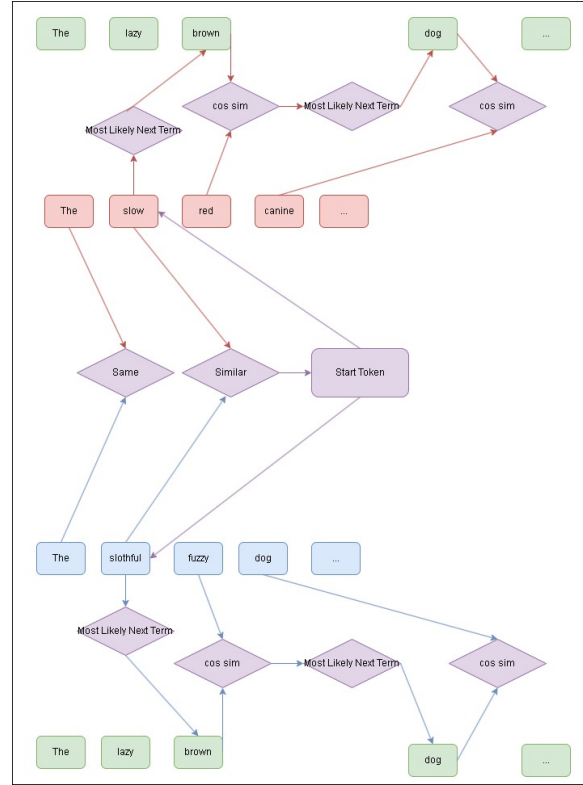


Figure 5. Beam search example

#### C. Term Frequency and Context Vectors:

These techniques can be applied to either the aforementioned methodologies, using term frequency the algorithm can predict the most important terms, which means when comparing, those terms are weighed heavier, which can lead to a more accurate similarity score.

#### D. Deep learning model:

Using a pretrained LLM, the last layers, can be re-trained on the Kaggle dataset. This approach is useful for understanding the semantic differences between the two groupings, listing and resume input. However, this comes at a cost with resources. This would be most useful for a chatbot, or some other version where a deeper understanding of the embedded text is more useful.

### VI. SYSTEM INTEGRATION INTO LARGER NETWORK:

#### A. Server System:

-Site: If the site is created from scratch, a Python script and agent can be added into the backend services directly, if the site is being built with another service, then a server would be needed to utilize the algorithm. This would be a peer-to-peer system architecture, where the host site would send a request with data for the algorithm to perform operations on.  
\*Website/Mobile Application: This would act very

similar to the site architecture, where a server would be needed to call the algorithm script to perform the tasks on the input resume.

\*Chatbot: If the use case is for a user to be able to communicate with the agent, then the agent needs to have a deeper semantic understanding of the user's resume, why a potential listing may be a good fit, or not. The temperature should be set as warm, as information should be accurate, however due to the users being vulnerable people, the chatbot should have a little more personality, or 'friendliness.' The best models to use would be a newer GPT LLM, running these would be computationally expensive, but would lend and produce the best results.

## VII. SUMMARY & ADDITIONAL THOUGHTS:

\*Temp Agency Setup: An interesting concept would be to create a temp agency, made up of screened local companies, that have a high use for temp agents. In this way, the agent would have access to a database of approved companies and would predict the best match of these locations, instead of, or in addition to, the job listings. -IT work: Many local companies require IT departments, which in turn use temp agencies to train and employ IT personnel. Examples would include many of the paper plants in Cartersville. -Hardware Repair/Warehouse Workers: Local companies that need workers for their warehouses, for loading/unloading delivery trucks, managing warehouses, hardware repair. These companies teach interns/temporary employees warehouse skills, as well as repairing equipment. Examples would include: Techquidation in Woodstock. -Detailing/Vinyl Wrapping: Small companies that focus on detailing cars. Higher pay than average with flexible hours. Some companies have travel vans, to go to customers, these companies teach how to clean vehicles, and wrap vehicles.

\*Training workshops: Use the similarity scores, and a LLM to discover the attributes a user is lacking for specific positions, to recommend training workshops that would focus on those specific skills, that are currently missing. \*Estimated Resources

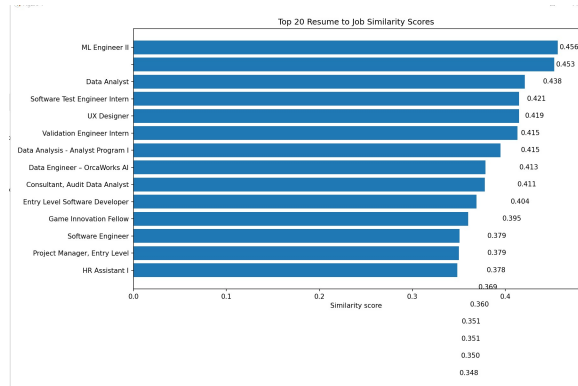


Figure 7. Results from embedded vector similarity

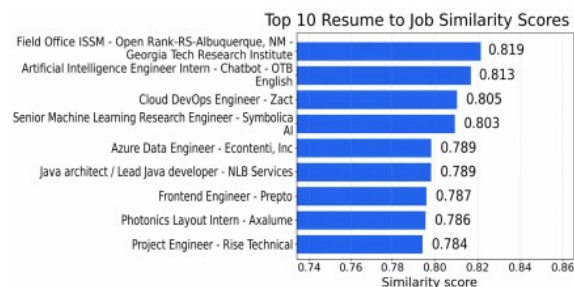


Figure 6. Results from learning model similarity